

Appian Placement Interview Experience

Pre-requisites: Python/Java/C++

Number of rounds: 4 (1 Online Assessment + 3 offline)

Online Assessment

3 coding questions

1. Breadcrumbs in a website

Given a list of list of string of paths to webpages in a website, generate the navigation tree and return the root node.

Sample Input:

```
[
["Root", "Profile", "Settings"],
["Root", "Profile", "Albums"],
["Root", "Settings"],
]
```

Sample Output:

```
Root
  Profile
    Settings
    Albums
  Settings
```

Code:

```
class TreeNode:
    def __init__(self, name):
        self.name = name
        self.children = []

    def add_child(self, child_name):
        for child in self.children:
            if child.name == child_name: #i.e, child already exists
                return child
        #else, create a tree node and append to self.child
        new_child = TreeNode(child_name)
        self.children.append(new_child)
        return new_child
```

```

def pretty_print(self, level=0):
    print('\t'*level + self.name + '\n')
    for child in self.children:
        child.pretty_print(level+1)

def construct_tree_from_breadcrumbs(breadcrumbs):
    root = TreeNode("Home")

    for breadcrumb in breadcrumbs:
        current_node = root
        for name in breadcrumb:
            current_node = current_node.add_child(name)

    root.children[0].pretty_print()

```

2. Given a line and a group registry which holds information on what group each person in line belongs to, re-organize the line such that
- if all the people of a particular group are present in line, then replace the first person of that group in line with their group name and remove the rest.
 - order of people in line shouldn't be changed for the rest

Sample Input:

```

line = [
    Person("Alice", "Group1"), Person("Bob", "Group1"), Person("Charlie", "Group2"),
    Person("David", "Group2"), Person("Eve", "Group3"), Person("Frank", "Group3"),
    Person("Eva", "Group4"),
]

```

Sample Output:

```
['Group1', 'Group2', 'Eve', 'Frank', 'Eva']
```

Code:

```

class Person:
    def __init__(self, name, group):
        self.name = name
        self.group = group

def reorganize_line(line, group_registry):
    group_dict = {group: [] for group in group_registry.keys()}
    group_order = []
    person_order = []

```

```

for person in line:
    if person.group in group_dict:
        if not group_dict[person.group]:
            group_order.append(person.group)
            group_dict[person.group].append(person.name)
        else:
            person_order.append(person.name)

final_list = []
for group in group_dict:
    if len(group_dict[group]) == len(group_registry[group]):
        final_list.append(group)
    else:
        final_list.extend(person for person in group_dict[group])
final_list.extend(person_order)

return final_list

```

3. There are n people waiting in line to buy tickets to a Taylor Swift concert. The ticket manager starts at an offset position m and distributes the first ticket. Then, the manager jumps k steps in a cyclic fashion to give the next ticket, and this process continues until all tickets are distributed. The task is to determine the order in which tickets are given out.

This sum is derived from the josephus sum.

Sample Input:

$n = 20$ (total number of people)
 $m = 14$ (starting offset position)
 $k = 4$ (number of steps to jump)

Sample Output:

[14, 18, 3, 7, 11, 15, 4, 8, 12, 16, 19, 2, 6, 10, 13, 17, 1, 5, 9, 0]

Code:

```

def print_lot_order(num_of_lots, step, offset):
    lottery_list = []
    for i in range(num_of_lots):
        lottery_list.append(i)

    #first draw
    start_index = (offset + step)%num_of_lots
    print(str(lottery_list.pop(start_index)),end=","")
    num_of_lots -= 1

    #subsequent draws
    while(num_of_lots>0):
        start_index = (start_index + step)%num_of_lots
        print(str(lottery_list.pop(start_index)),end=","")

```

```
num_of_lots -= 1

print_lot_order(20,4,14)
```

16 were shortlisted for the interview.

First Technical Round (40-45 minutes):

1. Coding question – Given a set of numbers, print the Fibonacci numbers alone.

Method 1:

I gave brute force answer for this and optimized it as much as possible. I iterated through the list and found whether it's a fibonacci number or not using an infinite loop, broke the loop when the Fibonacci number crossed the given number and returned true or false (i.e, whether a fibonacci number or not). Optimized by employing more breaking conditions in the initial iterative loop to reduce complexity and created an array to keep track of previously calculated Fibonacci values to reduce repetitive calculations. Was asked to calculate time complexity for this approach.

Method 2:

N is a Fibonacci number if and only if $(5*N^2 + 4)$ or $(5*N^2 - 4)$ is a perfect square. We can iterate through each element in the list and find out whether it satisfies the condition. Lower complexity than Method 1.

Then resume related questions were asked. Questions about my projects were asked. Difficulties faced during projects.

Second Technical Round (40-45 minutes):

1. Coding Question – Given a string, determine whether one or more of its permutations would result in a palindrome.

A stack can be used to solve this with ease. Push the characters in the stack and when you meet a character that's already been pushed, pop it out. By the end of the loop, the length of stack must be 1 or 0. If so, then return true.

Then Resume related questions were asked, this round focused on my intern experience. My intern work was discussed and questions related to it were asked. Like directives in angular (ngFor, ngIf, ngOptimizedImage etc. and their updated versions - @if, @for)

HR Round (20-25 minutes):

Just a normal conversation about my school and college performance. I asked about their work culture etc.

2 were selected from CSE for Software Engineering FTE.

Key Takeaways:

1. Connect with seniors who work there prior to the interviews (very important)
2. Be strong in Java (you can give solutions in any language, but knowledge in java is expected)
3. Be thorough with what you've done in your projects and deploy them.
4. Have a good CGPA.
5. Skills mentioned in the resume must be known.
6. Always have questions to ask the interviewers. You can ask about the culture, their favourite thing about the company etc. Also communicate openly when you aren't able to understand something.
7. As you code, communicate to the interviewer your thought process behind it, which will be very effective for them to understand your work and suggest improvements.
8. Research on the company and know about their core values.
9. Have a neat and precise resume. Include only the things you are thorough with. Do not overcrowd your resume.
10. Don't feel pressured to give an optimized answer right away. You can propose brute force approach and improve it from there.

Contact Information:

Email: vachu2224@gmail.com

LinkedIn: <https://www.linkedin.com/in/varsha-mohan-93913224a>

All the very best!! 😊